

CodeFlow: Intuitive Codebase Maps

Living documentation for your codebase

Cooper Keenan

Developers spend more time reading unfamiliar code than writing it, yet the diagrams meant to help show **structure** — which file imports which — rather than **behaviour**. A call graph answers "who calls whom". It does not answer "what happens when a request arrives".

CodeFlow takes a Python repository and produces a single-page diagram of what the system does when it runs: where requests enter, which branches genuinely change the outcome, and what the system writes to. Deterministic static analysis owns every node and edge; a temperature-zero language model only judges which branches matter and names them in human language. The pipeline is byte-identical across runs and every node carries a `file:line` reference, so a structural hallucination is impossible by construction rather than merely unlikely.

1 Introduction

The problem. Comprehension is the dominant cost in software work. Onboarding amplifies it, and written documentation rots away from the code it describes.

The gap. Comment-based generators need annotations that nobody writes. Tools such as PlantUML and Structurizr need the diagram authored by hand, so it is only ever as current as its author. Tools that hand a whole repository to a language model produce output that is plausible and confidently wrong, because nothing constrains the model to structure that actually exists.

Aim. Given only a repository, produce an accurate, readable, single-page diagram of what the system does when it runs.

Objectives

- Derive structure deterministically by static analysis, with no model influence over nodes or edges.
- Identify architecturally meaningful branch points and separate them from trivial guards.
- Render a bounded one-page view — entry, decision, effect — with services as swimlanes and cross-service calls stitched into one journey.
- Constrain the model to judging and naming only, validated against ids the pipeline issued.
- Deliver a usable product: web UI, GitHub sign-in, persistence, CI integration.
- Evaluate accuracy, determinism and utility.

2 Method: architecture

A FastAPI gateway, five agent services and a React frontend, with a typed contract on every hop, so each stage can be tested and replaced on its own. The tracer runs eight stages over the source tree, and only one of them — judging which forks are real — ever calls a model.

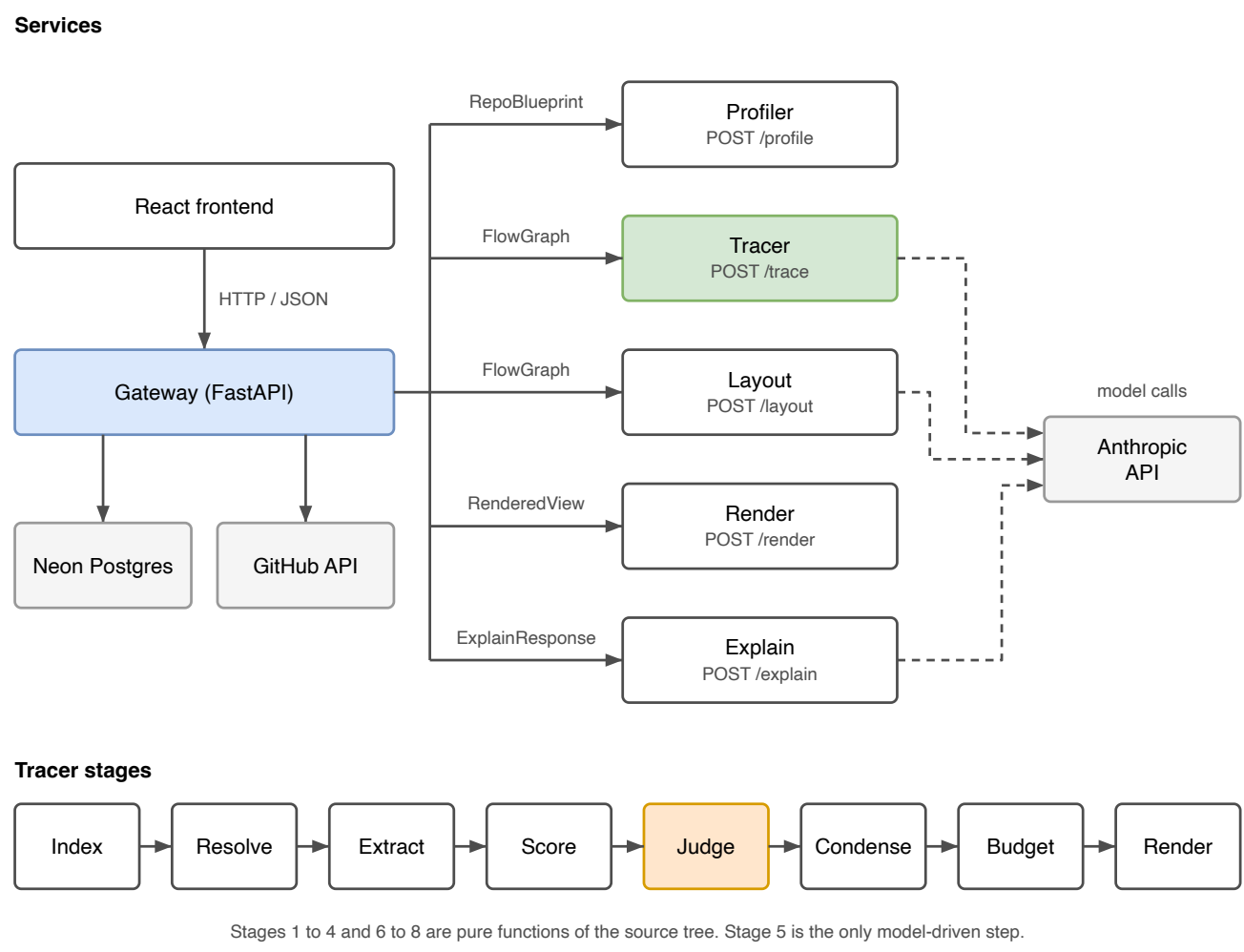


Figure 1 — System architecture. Dashed lines are the only paths that reach a model.

3 Method: which branches matter

Most branches in real code are noise. The hard problem is telling a **decision** — a fork that sends the system somewhere genuinely different — from a **guard** that checks a value and rejoins the same path.

```
if s is None:
```

Guard. Both arms rejoin the same work. Not something a person would draw.

```
if s == "cat":
```

Decision. The arms reach materially different downstream work.

The test is **reach-set mutual exclusivity**: compute what each arm can reach, and admit the fork only when the arms fail to reconverge. Seven detectors find candidate forks — branch, match, except, table, route, polymorphic and dynamic dispatch. Dispatch that cannot be resolved is drawn as dynamic, never guessed.

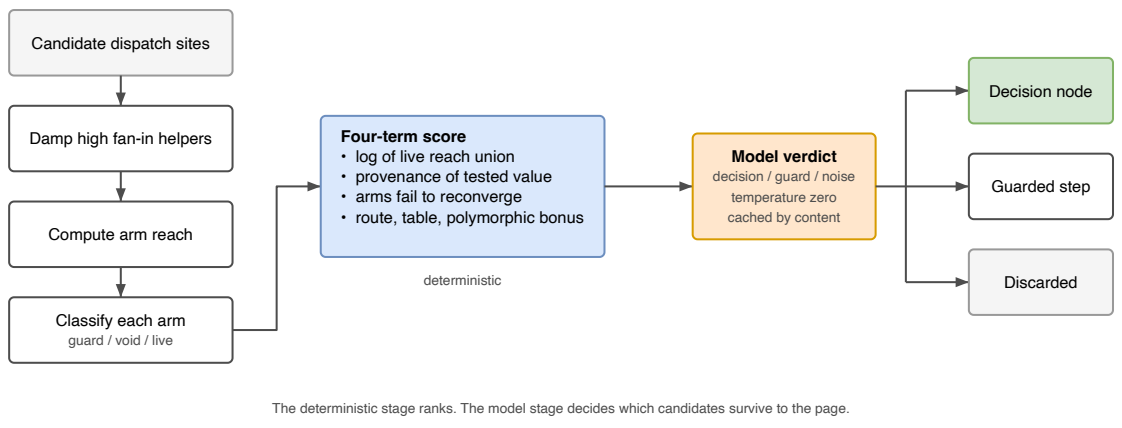


Figure 2 — The significance filter. Static analysis ranks every candidate; the model decides which survive to the page. **Assumption:** the weights and thresholds are tuned empirical constants, so the results below hold for this configuration.

5 What it produces

— every screenshot below is CodeFlow's own output on django-helpdesk, an unfamiliar third-party repository

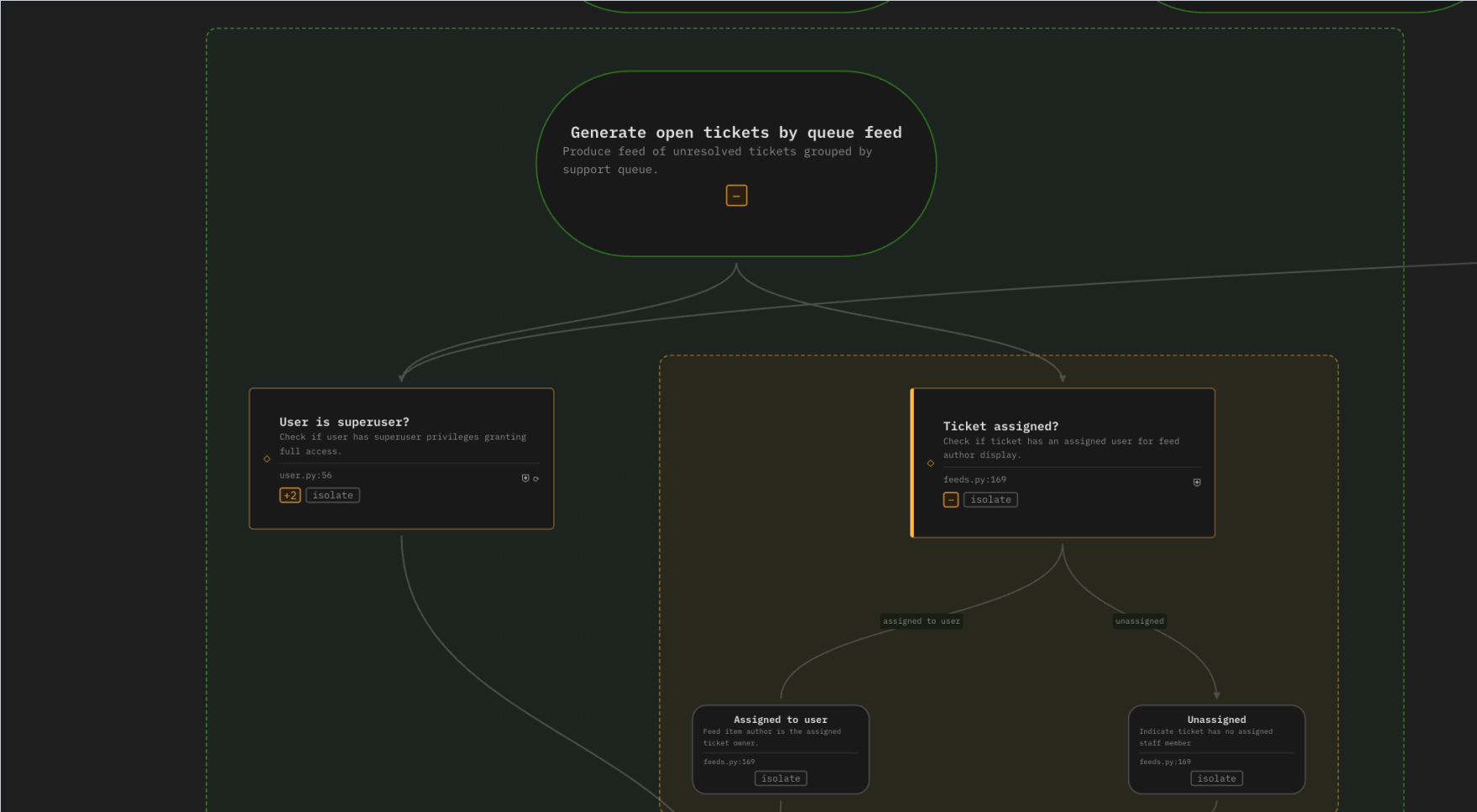


Figure 3 — The decision map. One entry expands to the decisions behind it: the questions the model wrote, and where each arm ends.

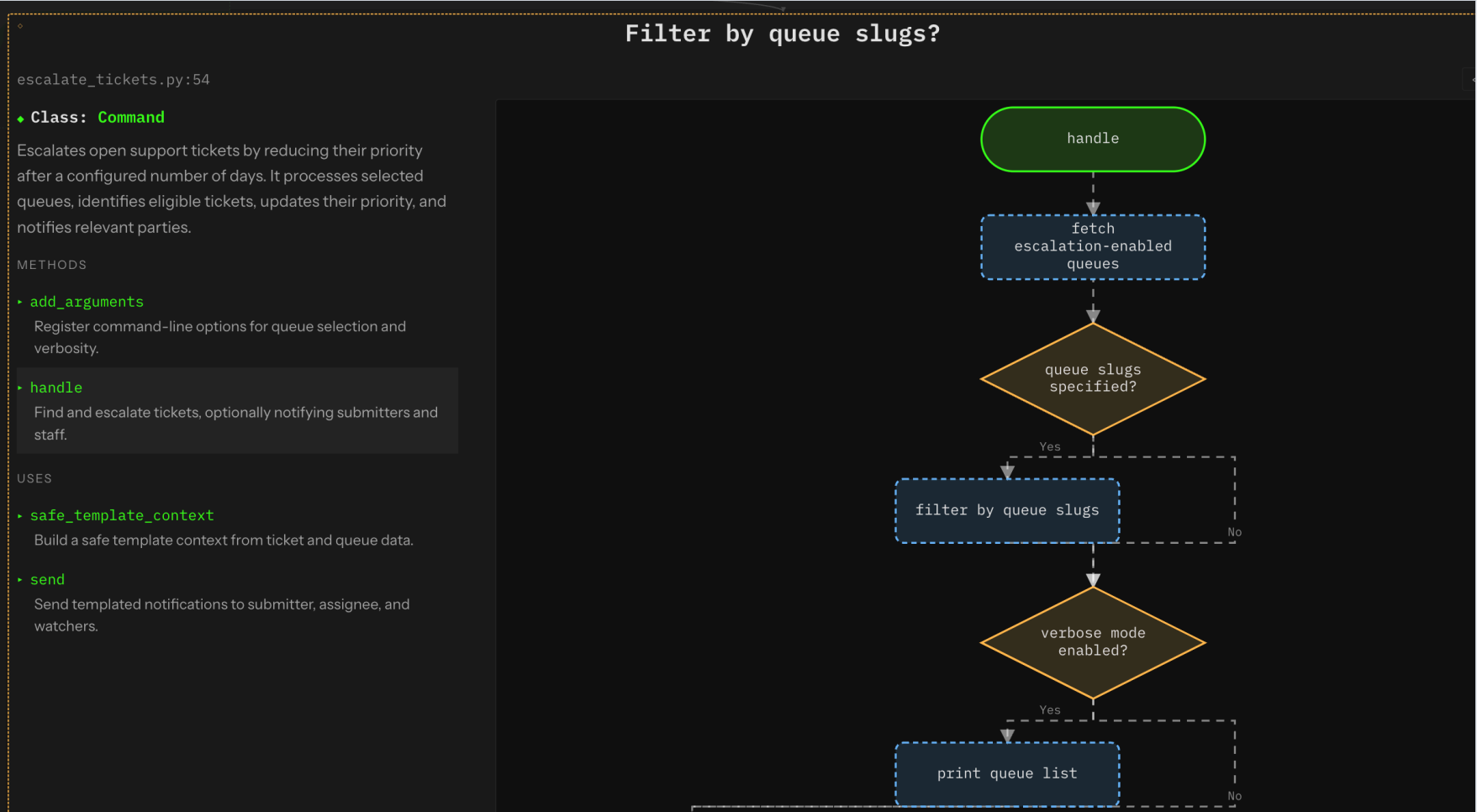


Figure 4 — Drill into one symbol. Any node opens as a local flowchart of the method itself, beside a plain-English summary of the class, its methods and what it uses.

```
escalate_tickets.py:54
```

```
> Class: Command
```

Escalates open support tickets by reducing their priority after a configured number of days. It processes selected queues, identifies eligible tickets, updates their priority, and notifies relevant parties.

METHODS

- `add_arguments`
Register command-line options for queue selection and verbosity.
- `handle`
Find and escalate tickets, optionally notifying submitters and staff.
- `safe_template_context`
Build a safe template context from ticket and queue data.
- `send`
Send templated notifications to submitter, assignee, and watchers.

USES

```
Filter by queue slugs?
```

```
1 def handle(self, *args, **options):
```

```
2     verbose = options["escalate_verbosely"]
```

```
3     notify_only = options["notify_only"]
```

```
4
```

```
5     queue_slugs = options["queues"]
```

```
6     # Only include queues with escalation configured
```

```
7     queues = Queue.objects.filter(escalate_days__isnull=False).exclude(
```

```
8         escalate_days=0
```

```
9     )
```

```
10    if queue_slugs is not None:
```

```
11        queues = queues.filter(slug__in=queue_slugs)
```

```
12
```

```
13    if verbose:
```

```
14        self.stdout.write(f"Processing: {queues}")
```

```
15
```

```
16    for queue in queues:
```

```
17        last = timezone.now().date() - timedelta(days=queue.escalate_days)
```

```
18        today = timezone.now().date()
```

```
19        workdate = last
```

```
20
```

```
21        days = 0
```

```
22
```

```
23        while workdate < today:
```

```
24            if not EscalationExclusion.objects.filter(date=workdate).exists():
```

```
25                days += 1
```

```
26                workdate = workdate + timedelta(days=1)
```

```
27
```

```
28        req_last_esc_date = timezone.now() - timedelta(days=days)
```

Figure 5 — Down to the source. The same frame toggles to the method body itself, so any claim the diagram makes can be checked against the code.

Reading the diagram

- Entry**
Where a request enters
- Decision**
A fork that changes the outcome
- Outcome**
Where one arm ends
- Step**
Work on the path between them

Colour is the node kind, not a highlight. Every node prints the `file:line` it came from; dashed containers mark what a single + revealed.

6 Results

identical	Determinism Two full runs over CodeFlow itself produce byte-identical graphs once labels are excluded.
463/499	Grounding Nodes carrying a <code>file:line</code> reference. The 36 without are entry seeds, which have no single call site by construction.
394 → 18	One page A 394-node graph of django-helpdesk opens as 18 nodes; the rest is revealed on demand.
0 / 0 / 0	Layout invariants Overlapping boxes, unreachable nodes and cohesion violations, on that same graph.
5,916 candidate forks judged	

63.4% guard	27.3% decision	9.3%
-------------	----------------	------

Nearly three quarters of candidate forks are rejected before they reach the page. Mean judge confidence 0.92.

7 Conclusion

Objectives 1, 2, 3 and 5 are met. Objective 4 was **revised rather than met**: the model was meant to name only, but separating decisions from guards proved to need judgement that no heuristic reproduced, so the model was given that verdict too — still with no power to add structure. Objective 6 is **partially met**: determinism and grounding are measured, but accuracy against a hand-derived ground truth, and comparison against a raw call graph and a one-shot model baseline, are designed and not yet run — so **no accuracy figure is claimed here**.

Limitations

- Tracing is Python-only, so the React frontend is invisible and no `frontend → api` call is ever stitched.
- The page budget is apportioned by lane mass, letting a route-heavy service crowd others out.
- CodeFlow itself is nearly linear, so the self-run under-sells the feature — which is why the figures above use a third-party repository.

8 Further work

- Cross-repository maps**, so a call that leaves one service can be followed into the next instead of stopping at the repository boundary.
- Infrastructure as code**: read a Terraform or Pulumi repository and draw the resources it provisions, beside the code that provisions them.
- Editable diagrams** (in progress), letting a reader correct or annotate a map and keep the correction.
- Change proposals**: draw a change before writing it, so the diagram becomes the brief you hand to a language model.